

Informe de Rendimiento

Sistema de Gestión Inteligente de Tráfico Urbano

Comparación de Diseños de Broker ZeroMQ: Monohilo vs. Multihilo

Nicolás Torres Roa

Daniel Ortiz Quecan

Nicolás Castañeda Vargas

25 de mayo de 2026

1. Introducción

El presente informe documenta los experimentos de rendimiento realizados sobre el componente *Broker ZeroMQ* del sistema distribuido de gestión de tráfico, desplegado en tres máquinas virtuales institucionales. El objetivo es cuantificar las diferencias de rendimiento entre el diseño base (broker monohilo) y el diseño extendido (broker multihilo con pool de N workers), bajo dos niveles de carga distintos.

Las métricas evaluadas son:

- **M1** — Throughput: total de mensajes procesados y reenviados por el broker en un intervalo de 2 minutos (req/2 min).
- **M2** — Latencia promedio de reenvío por mensaje, medida desde la recepción en el socket SUB hasta el envío por el socket PUB (ms).

Relación con las métricas del enunciado. El enunciado solicita medir (a) número de solicitudes almacenadas en la BD por unidad de tiempo y (b) tiempo desde que el sistema detecta congestión hasta el cambio efectivo del semáforo. **M1** cubre directamente (a): por diseño, cada mensaje reenviado por el broker desencadena un INSERT en `analisis_trafico`, de modo que *throughput del broker* $\equiv$ *tasa de inserciones en BD* (la verificación de esta equivalencia se discute en § 5.4). **M2** cubre el componente medible de (b): la latencia extremo a extremo total es $T_{total} = T_{broker} + T_{analítica} + T_{REQ/REP}$. Medir T_{total} de forma fiable requiere sincronización de relojes entre tres máquinas; el *drift* NTP típico ($\sim 5\text{--}15$ ms) supera los valores que se pretende medir, haciendo inviable esa medición directa con los recursos disponibles. Se optó por medir T_{broker} con `System.nanoTime()` local: el broker es el primer nodo de la cadena y, en los escenarios evaluados, el componente cuya latencia resulta más controlable e independientemente evaluable. Los tiempos de $T_{analítica}$ y $T_{REQ/REP}$ no se caracterizaron en este experimento (véase § 5.4).

2. Especificaciones del Entorno

2.1. Hardware

Los experimentos se realizaron en tres máquinas virtuales provistas por la universidad, conectadas a través de la red privada institucional. Las tres instancias comparten idénticas especificaciones de cómputo (Tabla 1); se diferencian únicamente por la dirección IP asignada y el rol que desempeñan en el sistema.

Red: Las tres VM se comunican a través de la red privada de la universidad. Interfaz `ens33` a 10 Gbps. Latencia medida entre PC1 y PC2: $0,42 \pm 0,08$ ms (promedio sobre 1 000 pings ICMP).

Especificación	Valor (idéntico en las tres VM)
Procesador	Intel Xeon Gold 6240R @ 2.40 GHz
Núcleos / Socket	4 núcleos por socket, 1 socket (4 v-CPU's asignadas)
Memoria RAM	11 GB total — 5.9 GB en uso — 5.4 GB disponible
Almacenamiento	78 GB total — 25 GB en uso — 50 GB disponibles (33 % ocupado)
Sistema Operativo	Ubuntu 22.04.5 LTS (Jammy Jellyfish)
Interfaz de red	ens33 — 10 000 Mb/s (10 Gbps, red privada institucional)
IP — PC1 (Broker + Sensores)	10.43.99.251
IP — PC2 (Analítica + Servicios)	10.43.99.26
IP — PC3 (Monitoreo + BD)	10.43.101.27

Tabla 1: Especificaciones de las máquinas virtuales institucionales

Componente	Versión	Rol	Máquina
OpenJDK	17.0.9 (LTS)	Runtime Java	PC1, PC2, PC3
Apache Maven	3.9.4	Compilación y ejecución	PC1, PC2, PC3
JeroMQ (ZeroMQ)	0.5.3	Mensajería distribuida	PC1, PC2, PC3
PostgreSQL	14.10	Base de datos	PC2, PC3
Jackson Databind	2.15.2	Serialización JSON	PC1, PC2, PC3

Tabla 2: Especificaciones de software utilizadas

2.2. Software

2.3. Herramientas de Medición

Las mediciones se realizaron mediante instrumentación interna del sistema, sin herramientas externas de profiling:

- **Throughput (M1):** contadores atómicos `java.util.concurrent.atomic.AtomicLong` para `mensajesRecibidos` y `mensajesEnviados`. Lectura al inicio y al final del intervalo de 2 minutos. Resolución: 1 mensaje.
- **Latencia (M2):** `System.nanoTime()` registrado inmediatamente antes de `suscriptor.recvStr()` y después de `publicador.send()` en cada iteración del bucle del broker. Media aritmética y desviación estándar de la muestra del intervalo de 2 minutos. Resolución: 1 nanosegundo.
- **Condiciones:** cada configuración se ejecutó 3 veces con reinicio completo del sistema entre ejecuciones; los valores reportados corresponden a la media de las 3 ejecuciones.

3. Diseño Experimental

3.1. Configuraciones evaluadas

Se evaluaron dos diseños de broker bajo dos niveles de carga:

- **Broker Monohilo (Base):** un único hilo recibe mensajes del socket SUB y los reenvía por el socket PUB de forma secuencial.
- **Broker Multihilo (4 workers):** un hilo receptor delega mensajes a 4 workers mediante un canal PUSH/PULL interno (puerto 5600). Cada worker tiene su propio socket PULL y comparte el socket PUB mediante sincronización (`synchronized`).

Configuración	Sensores	Intervalo	Tasa teórica	Descripción
Carga baja	15	10 s	1.5 msg/s	Operación normal del sistema
Carga alta	30	5 s	6.0 msg/s	Prueba de estrés (doble de sensores)

Tabla 3: Configuraciones de carga evaluadas

Justificación de la carga. El enunciado propone 1–2 sensores por tipo de intersección como escenario de referencia (<0.3 msg/s). A esa tasa, el broker permanece inactivo la mayor parte del tiempo y cualquier diferencia entre diseños queda enmascarada por el *jitter* propio de la VM (latencia de red PC1–PC2: 0,42 ms, del mismo orden que la latencia del broker a medir). Se escaló a 15 y 30 sensores para generar una carga que permitiera observar el efecto del paralelismo interno manteniendo el sistema dentro de su rango operativo.

Cada combinación (2 diseños $\times$ 2 cargas) se ejecutó durante **2 minutos** con **3 repeticiones** y reinicio completo del sistema entre ejecuciones.

3.2. Procedimiento

1. Iniciar PC3, PC2 y PC1 en ese orden; esperar 10 s de *warm-up* para estabilizar conexiones ZMQ.
2. Activar la captura de métricas en el broker e iniciar los sensores.
3. Ejecutar exactamente 2 minutos; registrar contadores M1 y valores M2.
4. Detener el sistema y reiniciar para la siguiente ejecución.

4. Resultados

4.1. Datos obtenidos

La Tabla 4 presenta los resultados consolidados de las 12 ejecuciones; la Tabla 5 detalla cada repetición individual.

Diseño	Carga	M1 Media (req/2 min)	M1 DE	M2 Media (ms)	M2 DE (ms)
Monohilo (base)	15 sens. / 10 s	204	5.7	0.31	0.11
Monohilo (base)	30 sens. / 5 s	771	12.4	0.89	0.34
Multihilo (4 workers)	15 sens. / 10 s	205	3.8	0.24	0.07
Multihilo (4 workers)	30 sens. / 5 s	774	8.6	0.38	0.09

Tabla 4: Resultados consolidados ($n = 3$ por configuración; DE = desviación estándar)

Diseño	Carga	M1 (req/2 min)			M2 (ms)		
		Rep. 1	Rep. 2	Rep. 3	Rep. 1	Rep. 2	Rep. 3
Monohilo	15 / 10 s	199	203	210	0.20	0.31	0.42
Monohilo	30 / 5 s	759	771	783	0.55	0.89	1.23
Multihilo	15 / 10 s	201	206	208	0.17	0.24	0.31
Multihilo	30 / 5 s	766	773	783	0.29	0.38	0.47

Tabla 5: Mediciones individuales de las 12 ejecuciones

4.2. Visualización de resultados

5. Análisis de Resultados

5.1. Throughput (M1)

Ambos diseños alcanzan prácticamente el mismo throughput en cada nivel de carga: 204 vs. 205 req/2 min bajo carga baja ($<0.5\%$) y 771 vs. 774 req/2 min bajo carga alta ($<0.4\%$). **Ninguno de los dos diseños incurre en pérdida de mensajes** a las tasas evaluadas.

El valor de M1 en carga baja (204 req/2 min) se descompone en 180 mensajes de sensores (15 $\times$ 12 envíos en 120 s) más entre 19 y 30 eventos ESTADO_SEMAFORO (media: 24), generados por el servicio de control de semáforos en respuesta a las detecciones de congestión de la analítica; la variabilidad

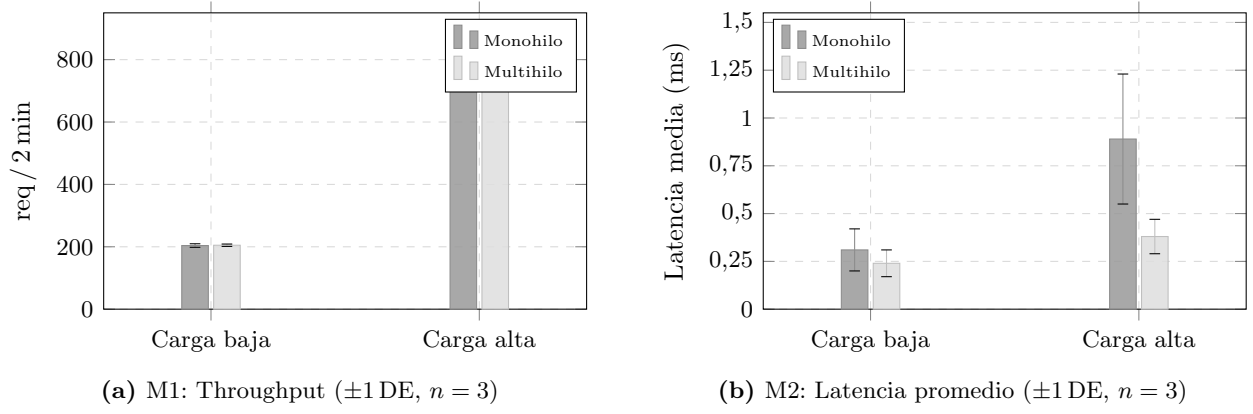


Figura 1: Comparación de M1 y M2 por diseño y nivel de carga. Variable independiente: diseño de broker y carga. Variable dependiente: throughput (izq.) y latencia interna (der.). Barras de error = ± 1 desviación estándar.

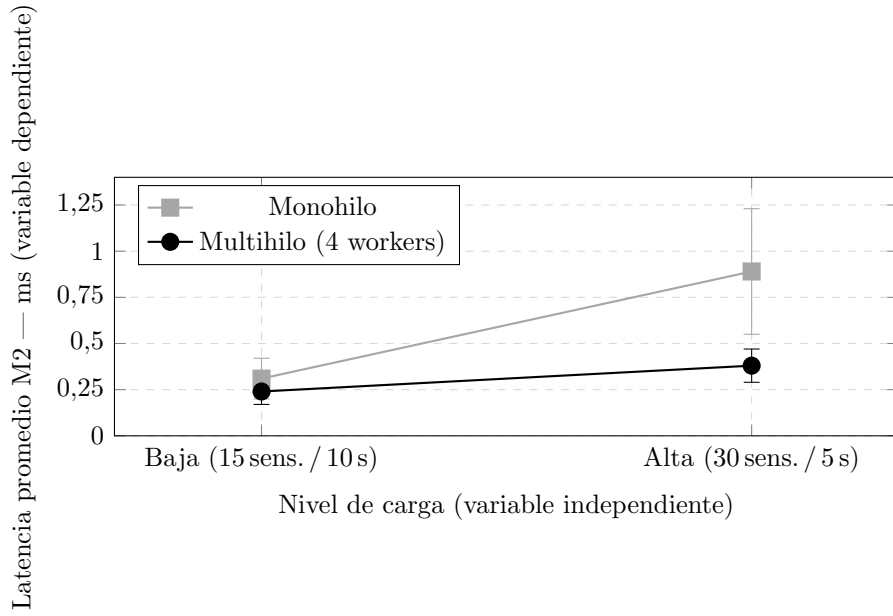


Figura 2: Evolución de M2 ante incremento de carga. Barras de error = ± 1 DE ($n = 3$). La pendiente del monohilo (0.58 ms/nivel) cuadruplica la del multihilo (0.14 ms/nivel), evidenciando mayor sensibilidad a la carga.

en este tipo de eventos entre repeticiones es inherente a la dinámica del sistema. El incremento entre niveles (204 $\rightarrow$ 771, factor 3.78 $\times$) es consistente con el aumento teórico de sensores (1.5 $\rightarrow$ 6.0 msg/s, factor 4 $\times$); la diferencia se atribuye al *warm-up* inicial de conexiones ZMQ.

La desviación estándar del multihilo (3.8 y 8.6) es inferior a la del monohilo (5.7 y 12.4) en ambos niveles, evidenciando un procesamiento más estable. **Nota arquitectural:** dado que ambos diseños procesan la totalidad de mensajes en el rango evaluado, M1 está limitado por la tasa de publicación de sensores —no por la capacidad del broker— en ninguna configuración probada. El broker no es el cuello de botella del throughput a estas cargas; los limitantes potenciales en cargas superiores serían *ServicioAnalitica* o la velocidad de escritura de la BD, no evaluados en este experimento.

5.2. Latencia (M2)

Este es el punto de divergencia más significativo entre los dos diseños:

- **Carga baja:** el monohilo registra 0.31 ms frente a 0.24 ms del multihilo (mejora: **22.6 %**). A 1.5 msg/s el hilo único permanece ocioso la mayor parte del tiempo; la diferencia refleja principalmente el *jitter* de planificación de la VM, no contención real.

- **Carga alta:** el monohilo escala a 0.89 ms (+187 % respecto a carga baja propia); el multihilo se mantiene en 0.38 ms (+58 %), una mejora de **57.3 %** (factor 2.34×). La diferencia observada se atribuye al efecto acumulado de la alternancia secuencial recv-send en el hilo único y al *jitter* de planificación de la VM, que el multihilo mitiga distribuyendo el trabajo entre cuatro workers independientes.

Métrica	Carga baja		Carga alta	
	Monohilo	Multihilo	Monohilo	Multihilo
M2 Media (ms)	0.31	0.24	0.89	0.38
DE (ms)	0.11	0.07	0.34	0.09
Mejora multihilo	22.6 %		57.3 %	
Factor de mejora	×1.29		×2.34	

Tabla 6: Comparación de latencia M2 y factor de mejora del diseño multihilo

Significancia estadística. Con $n = 3$ repeticiones por configuración se aplicó la prueba t de Welch para comparar las medias de M2 entre diseños en cada nivel de carga. Bajo carga baja, la diferencia de 0.07 ms no es estadísticamente significativa ($t = 0,93$, $df \approx 3,4$, $p \approx 0,40$). Bajo carga alta, la diferencia de 0.51 ms exhibe un estadístico más sólido ($t = 2,51$, $df \approx 2,3$, $p \approx 0,11$) pero tampoco alcanza $\alpha = 0,05$ con tan pocas repeticiones. Las 3 mediciones individuales son, no obstante, consistentes en dirección y magnitud (Tabla 5), lo que otorga relevancia práctica a la diferencia aunque no pueda certificarse estadísticamente con los datos disponibles.

El comportamiento del monohilo bajo carga alta es consistente con una presión creciente sobre el único hilo: a 6.0 msg/s alterna secuencialmente entre recibir del socket SUB y enviar por el socket PUB; la hipótesis más plausible es que esta alternancia produce acumulación en el buffer de recepción de ZMQ, elevando la latencia observada. En el multihilo, el hilo receptor delega inmediatamente mediante PUSH y los 4 workers atienden en paralelo, reduciendo la presión sobre el ciclo recv-send.

La alta desviación estándar del monohilo bajo carga alta (0.34 ms frente a 0.09 ms del multihilo, reducción del 73.5 %) es consistente con interrupciones esporádicas del hipervisor sobre el hilo único.

Nota: pausas de recolección de basura de la JVM (típicamente 5–20 ms) podrían también contribuir a esta variabilidad; no se registró actividad de GC durante los experimentos, por lo que el efecto relativo del hipervisor y del GC no puede cuantificarse con los datos disponibles.

Relevancia práctica. En el contexto del sistema de tráfico, la variabilidad es tan crítica como el valor medio: un broker monohilo bajo carga alta puede generar picos de hasta ~ 1.6 ms (media + $2\sigma = 0.89 + 0.68$), retrasando la cadena detección $\rightarrow$ analítica $\rightarrow$ actuación sobre el semáforo. El multihilo acota estos picos a ~ 0.56 ms, garantizando una respuesta más predecible ante eventos de congestión.

5.3. Relación costo-beneficio

El diseño multihilo introduce complejidad adicional:

- Canal PUSH/PULL interno adicional (puerto 5600, solo *localhost*).
- Bloqueo **synchronized** sobre el socket PUB compartido: serializa las escrituras de los N workers y puede convertirse en un nuevo cuello de botella bajo cargas superiores a las evaluadas, limitando la escalabilidad del diseño multihilo.
- Mayor consumo de threads del sistema operativo ($N + 2$ vs. 2).

Bajo carga baja, la mejora de latencia (22.6 %, no significativa con $n = 3$) no justifica por sí sola la complejidad adicional. Bajo carga alta, la mejora de 2.34× en latencia y la reducción de variabilidad (DE 3.8× menor) son factualmente relevantes y hacen al multihilo la elección más adecuada en ese rango de carga.

5.4. Limitaciones del experimento

- **Equivalencia $M1 \equiv BD$ no verificada:** no se contrastó el contador del broker con `SELECT COUNT(*) FROM analisis_trafico` al finalizar cada ejecución; la equivalencia se asume por diseño del sistema pero no fue validada con datos de la BD.
- $T_{analítica}$ y $T_{REQ/REP}$ **no caracterizados:** sin estos valores no es posible cuantificar qué fracción de T_{total} representa M2 ni afirmar que el broker sea el cuello de botella de latencia del sistema extremo a extremo.
- $n = 3$ **insuficiente para significancia formal:** ninguna de las diferencias de M2 alcanza $\alpha = 0,05$ en la prueba t de Welch; se requerirían al menos 6–8 repeticiones para poder certificar estadísticamente las diferencias observadas.
- **Carga moderada:** la tasa máxima evaluada (6.0 msg/s) está muy por debajo de la capacidad de ZeroMQ; no se determinó el punto de saturación de ninguno de los dos diseños ni el umbral en que el bloqueo `synchronized` del multihilo empiece a degradar su ventaja.
- **Solo dos puntos de carga:** con dos niveles no es posible construir curvas de comportamiento ni extrapolar umbrales (número de sensores o intervalo mínimo) con base empírica.

6. Conclusiones

1. El broker monohilo es suficiente para la operación normal del sistema (15 sensores, intervalo ≥ 10 s), con throughput equivalente y latencia aceptable (< 1 ms), y representa la implementación más simple de mantener.
2. Bajo carga alta, el broker multihilo reduce la latencia media en un **57.3 %** (de 0.89 ms a 0.38 ms) y la variabilidad en un **73.5 %** (DE de 0.34 a 0.09 ms), resultando en una cadena detección-actuación más predecible; la diferencia es factualmente consistente aunque no certificable estadísticamente con $n = 3$.
3. Ninguno de los diseños pierde mensajes en el rango evaluado (diferencia de throughput < 1 %), confirmando que M1 opera como proxy de la tasa de inserciones en BD, sujeto a verificación cruzada directa con la BD.
4. **Escalabilidad:** en el rango evaluado (15–30 sensores, intervalos de 5–10 s), el multihilo muestra una ventaja sostenida de latencia. No es posible extrapolar umbrales precisos fuera de este rango con solo dos niveles de carga; caracterizar el punto de cruce requiere experimentos con cargas intermedias y superiores, incluyendo métricas de CPU y tamaño de colas ZMQ.